第3章　运算方法与运算器

在计算机中，运算器用于数值运算及加工处理数据，它是由CPU中的算术逻辑单元、通用寄存器等部件构成。运算器的结构取决于指令系统、数据的表示方法、运算方法及所选用的硬件。本章的主要内容就是讨论数值运算的方法及其具体实现。

3.1　定点数运算

如第2章所述，计算机中常用定点数或浮点数这两种形式来表示数值。它们的运算方法也有所不同，下面将分别予以讨论。本节将描述定点数的四则运算的法则及实现方法。

3.1.1　加减运算

1. 加减运算方法

在第2章中，已经描述了数值的编码，它们可以用原码、反码、补码、移码等形式来表示。原则上讲，对数值的加减运算可以用上述任何一种编码来实现，但是，用得最多最普遍的还是采用补码来实现加减法。下面将说明补码加减运算法则及有关问题。

补码加减运算过程中，参加运算的操作数及运算结果均用补码表示。

(1) 补码加法

补码加法的运算法则为：

$$[X+Y]_{补}=[X]_{补}+[Y]_{补} \tag{3-1}$$

由式(3-1)可以看到，两数和的补码就等于两数的补码之和。利用补码求两数之和就变得十分方便。

【例3-1】 若两个定点整数63和35，利用补码加法求63+35=？

解：根据题意，用8位二进制补码表示63和35。

$$[63]_{补}=00111111$$

$$[35]_{补}=00100011$$

则$[63+35]_{补}=01100010$。

【例3-2】 若两个定点整数−63和−35，利用补码加法求−63+(−35)=？

解：根据题意，用8位二进制补码表示−63和−35。

$$[-63]_{补}=11000001$$

$$[-35]_{补}=11011101$$

则$[-63+(-35)]_{补}=10011110$。

(2) 补码减法

在数值的补码表示法中，对一个正数求补——即对该数包括符号位在内各位取反再加1，

即可得到该数的负数；若对该负数再求补又可得到原来的正数。也就是说$[[X]_{补}]_{求补}=[-X]_{补}$；$[[-X]_{补}]_{求补}=[X]_{补}$。

补码减法的运算法则为：

$$[X-Y]_{补}=[X]_{补}+[-Y]_{补}=[X]_{补}+[[Y]_{补}]_{求补} \tag{3-2}$$

式(3-2)给予一个最重要的启示就是用补码做减法时，可用加法来实现；被减数减去减数可化做被减数加上减数求补来完成。这一特征就使得 CPU 的设计者不必在 CPU 中设置减法器，只要设置一个加法器，就既能实现加法运算又能完成减法运算。这也是计算机中普遍的是采用补码来实现加减法的缘由。

【例 3-3】 若两个定点整数 63 和 35，利用补码减法求 63－35＝?

解：根据题意，用 8 位二进制补码表示 63 和 35。

$$[63]_{补}=00111111$$

$$[35]_{补}=00100011$$

而$[63-35]_{补}=[63]_{补}+[-35]_{补}$；同时，$[-35]_{补}=[[35]_{补}]_{求补}=11011101$，从而求出：

$$\begin{array}{r} 00111111 \\ +11011101 \\ \hline 1\quad 00011100 \end{array}$$

得到$[63-35]_{补}=00011100$。由于是 8 位二进制运算，在上面的相加过程中有一进位 1 丢弃不用，所得的结果仍是正确的。

综上所述，补码加减运算的规则如下。

① 参加运算的操作数用补码表示。

② 符号位参加运算。

③ 若相加，则两个数的补码直接相加；若进行相减运算，则将减数连同符号位一起变反加 1 后与被减数相加。

④ 运算结果用补码表示。

2. 溢出判断

(1) 溢出的概念

从上面的描述中已经明确，只用加法器就可以实现补码加法和补码减法。但在运算的过程中有可能会产生溢出。为了说明溢出的概念及其判别方法，首先来看下面的例子。

【例 3-4】 若两个定点整数 63 和 85，利用补码加法求 63＋85＝?

解：根据题意，若用 8 位二进制补码表示 63 和 85。

$$[63]_{补}=00111111$$

$$[85]_{补}=01010101$$

$$\begin{array}{r} 00111111 \\ +01010101 \\ \hline 10010100 \end{array}$$

由上面的计算可以看到，两个正数(63 和 85)相加的结果变成一个负数(符号位为 1)，这显然是荒谬的。出现这种错误结果是由于在相加的过程中产生了溢出。原因就在于运算的结果已超出所规定的数值范围。如上所述，规定用 8 位二进制补码来表示带符号的整数，它所能表示的数值范围是－128～＋127。如果运算的结果超出了＋127 则称为上溢出；若

超出－128称为下溢出。

一旦确定了运算的字长和数据的表示方法后，数据的范围也就确定了。只要运算结果超出所能表示的数据范围，就会发生溢出。发生溢出时，运算结果肯定是错误的。只要发现运算结果产生溢出，就必须采取措施防止溢出发生。最简单有效的方法就是增加补码的二进制编码长度。在上面的例子中，只要将数值编码位数增加，如采用16位编码，就能防止溢出的发生。

值得注意的是，只有当两个同符号的数相加（或者是相异符号数相减）时，运算结果才有可能溢出。而在异符号的数相加（或者是同符号数相减）时，永远不会产生溢出。下面再举两例说明。

【例3-5】 设正整数 X＝＋1000001，Y＝＋1000011，若用8位补码表示，则$[X]_{补}$＝01000001，$[Y]_{补}$＝01000011，求$[X+Y]_{补}$。

解： 计算$[X]_{补}+[Y]_{补}$

$$\begin{array}{r} 0\ 1000001 \\ +\ \ 0\ 1000011 \\ \hline 1\ 0000100 \end{array}$$

两个正数相加的结果为一个负数，结果显然是错误的。产生错误的原因就是溢出。

【例3-6】 设负整数 X＝－1111000，Y＝－10010，若用8位补码表示，则$[X]_{补}$＝10001000，$[Y]_{补}$＝11101110，求$[X+Y]_{补}$。

解： 计算$[X]_{补}+[Y]_{补}$

$$\begin{array}{r} 1\ 0001000 \\ +\ \ 1\ 1101110 \\ \hline 0\ 1110110 \end{array}$$

两个负数相加，结果为一个正数，显然也是错误的。

在上面的举例中，所用的数据都是定点整数。上面的概念和结论对于定点纯小数也是完全适用的。但需注意所规定的定点纯小数的数值范围。

（2）溢出的判定

常用的溢出检测机制主要有进位判决法和双符号位判决法等如下几种方法。

① 双符号位（变形码）判决法

第2章中曾提到变形码，采用两位表示符号，即00表示正号、11表示负号，一旦发生溢出，则两个符号位就一定不一致，利用判别两个符号位是否一致便可以判定是否发生了溢出。

若运算结果两符号位分别用 S_2S_1 表示，则判别溢出的逻辑表示式为：

$$VF=S_2 \oplus S_1 \tag{3-3}$$

【例3-7】 设两正整数 X＝＋1000001，Y＝＋1000011，若用双符号位的8位补码表示，则$[X]_{补}$＝00 1000001，$[Y]_{补}$＝00 1000011，求$[X+Y]_{补}$。

解： 计算$[X]_{补}+[Y]_{补}$

$$\begin{array}{r} 00\ 1000001 \\ +\ 00\ 1000011 \\ \hline 01\ 0000100 \end{array}$$

式中，由于结果的 S_2 和 S_1 不一致，$VF = S_2 \oplus S_1 = 1$，溢出发生。

② 进位判决法

若 C_{n-1} 为最高数值位向最高位(符号位)的进位，C_n 表示符号位的进位，则判别溢出的逻辑表示式为：

$$VF = C_{n-1} \oplus C_n \tag{3-4}$$

溢出判别表如表 3-1 所示。

表 3-1 溢出判定表

C_{n-1}	C_n	$C_{n-1} \oplus C_n$	C_{n-1}	C_n	$C_{n-1} \oplus C_n$
0	0	0	1	0	1
0	1	1	1	1	0

在例 3-6 的运算过程中，C_{n-1} 为 0 而 C_n 为 1 ，故 $C_{n-1} \oplus C_n = 1$，表示运算结果有溢出。

③ 根据运算结果的符号位和进位标志判别

该方法适用于两同号数求和或异号数求差时判别溢出。溢出的逻辑表达式为：

$$VF = SF \oplus CF \tag{3-5}$$

式(3-5)中 SF 和 CF 分别是运算结果的符号标志和进位标志。

在使用式(3-5)判别溢出时，一定要注意该判别方法的适用条件，否则，将会出错。

④ 根据运算前后的符号位进行判别

若用 Xs、Ys、Zs 分别表示两个操作数及运算结果的符号位，当两同号数求和或异号数求差时，就有可能发生溢出。溢出是否发生可根据运算前后的符号位进行判别，其逻辑表达式为：

$$VF = Xs \cdot Ys \cdot \overline{Zs} + \overline{Xs} \cdot \overline{Ys} \cdot Zs \tag{3-6}$$

在 CPU 中，进行定点算术运算是否发生溢出，通常是由 CPU 中的硬件逻辑电路进行检测。一旦溢出发生，则会在 CPU 中的标志寄存器中建立溢出标志，或者产生溢出中断。

3. 一位全加器的实现

设一位全加器的输入分别为 X_i 和 Y_i，低一位对该位的进位为 C_i。全加器的结果和向高一位的进位分别用 Z_i 和 C_{i+1} 表示。则一位全加器所实现的逻辑表达式如下：

$$Z_i = X_i \oplus Y_i \oplus C_i \tag{3-7}$$

$$C_{i+1} = (X_i \cdot Y_i) + (X_i + Y_i) \cdot C_i \tag{3-8}$$

实现上述逻辑功能的一位全加器的逻辑电路及其框图分别如图 3-1(a)和(b)所示。

若令 $G_i = X_i \cdot Y_i$，$P_i = X_i + Y_i$，则式(3-8)可写为：

$$C_{i+1} = G_i + P_i \cdot C_i \tag{3-9}$$

人们将 G_i 称为生成进位的产生函数，而将 P_i 称为进位的传递函数。

4. n 位加法器的实现

(1) 行波进位加法器

利用上述 n 个全加器串联在一起工作，使可以构成 n 位加法器。同时，前面已经提到，补码加减运算，用加法器便可实现。下面图 3-2 所给出的就是用 n 个一位全加器及门电路构成 n 位补码加法/减法器的框图。

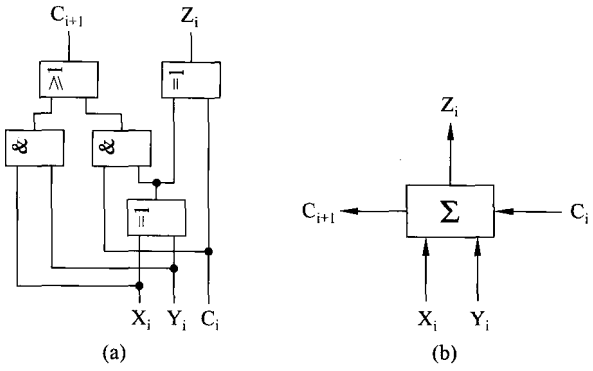


图 3-1 一位全加器逻辑及其框图

根据图 3-2，注意到如下几点。

① 在此加法器中，进位是逐位形成的。如图 3-1 所看到的那样，进位的产生是由三级门电路形成的，是需要花时间的。因为，任何一级门电路必定有延时。而图 3-2 所示的 n 位加法器中，高位运算必须等低运算的进位产生后方能进行。进位是由低到高逐位产生，一位接一位向高位传递。故这种加法器通常称为行波进位加法器。若一位全加器的进位延时为 Δt，则上述 n 位加法器的延时就是 $n\Delta t$。也就是说完成 n 位加法的时间需要 $n\Delta t$。应当明确，设计运算器(或 CPU)所追求的最重要的指标之一是速度。显然，行波进位加法器的速度是难以令人满意的。

② 在图 3-2 中，利用增加异或门和控制信号 M：当 M 为 0 时，实现加法器的功能；当 M 为 1 时，可将操作数 Y 的各位取反并在 C_0(最低位)上加上 M(此时是 1)，从而实现求补功能。然后再做加法——完成减法的功能。

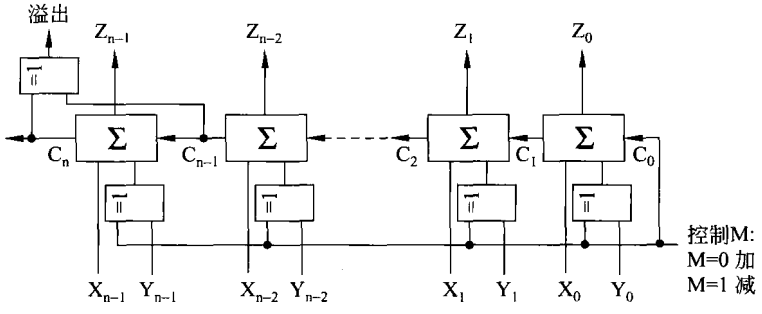


图 3-2 行波进位的 n 位加法/减法器

③ 在图 3-2 中，利用异或门完成式(3-4)所示的溢出判别功能。该异或门的输入刚好是 C_{n-1} 和 C_n。

(2) 先行进位加法器

综上所述，行波进位加法器尽管简单，但其致命的缺点就在于当加法器的位数增加时，由于进位而造成的加法速度的降低使人不能忍受。是否有加快进位速度的方法呢？答案是肯定的。

首先，分析式(3-9)，其表示式为：$C_{i+1}=G_i+P_i \cdot C_i$。从式中可知，只要有输入 X_i 和 Y_i 就能求出 G_i 和 P_i，在已知输入 C_i 的情况下，便可以获得 C_{i+1}。

那么，在有输入 X_{i+1}、Y_{i+1} 和 C_{i+1} 的情况下，便可以获得 C_{i+2}。以此类推，在输入存在的情况下，便可以求出 C_{i+3}、C_{i+4}…下面仅将 4 个进位的产生逻辑表示式写出。

$$C_{i+1}=G_i+P_iC_i \tag{3-10}$$

$$C_{i+2}=G_{i+1}+P_{i+1}C_{i+1}=G_{i+1}+P_{i+1}G_i+P_{i+1}P_iC_i \tag{3-11}$$

$$C_{i+3}=G_{i+2}+P_{i+2}C_{i+2}=G_{i+2}+P_{i+2}G_{i+1}+P_{i+2}P_{i+1}G_i+P_{i+2}P_{i+1}P_iC_i \tag{3-12}$$

$$C_{i+4}=G_{i+3}+P_{i+3}C_{i+3}=G_{i+3}+P_{i+3}G_{i+2}+P_{i+3}P_{i+2}G_{i+1}+P_{i+3}P_{i+2}P_{i+1}G_i+P_{i+3}P_{i+2}P_{i+1}P_iC_i=G^*_{i+3}+P^*_{i+3}C_i \tag{3-13}$$

式(3-13)中，

$$G^*_i=G_{i+3}+P_{i+3}G_{i+2}+P_{i+3}P_{i+2}G_{i+1}+P_{i+3}P_{i+2}P_{i+1}G_i$$

$$P^*_i=P_{i+3}P_{i+2}P_{i+1}P_i。$$

由上面的逻辑表达式(3-10)到式(3-13)可以看到，利用输入信号 X_i、X_{i+1}、X_{i+2}、X_{i+3} 和 Y_i、Y_{i+1}、Y_{i+2}、Y_{i+3} 以及 C_i，通过与或逻辑电路的组合就可以同时将上面 4 个信号产生出来。然后，将这些先行形成的进位信号并行地加到加法器上，则加法器就不必等待进位产生，从而大大地提高了加法器的速度。也就是说在进行加法运算之前，各位加法器所需要的进位都已经产生出来。这将加快加法运算的速度，这就是先行进位的由来。

根据上面的分析，具体用与或组合逻辑构成的 4 位先行进位产生电路如图 3-3 所示。

由图 3-3 的逻辑图可以看到，图 3-3 的下方是输入信号 X_i、X_{i+1}、X_{i+2}、X_{i+3} 和 Y_i、Y_{i+1}、Y_{i+2}、Y_{i+3} 以及 C_i。这些信号经过三级门电路便可以得到所有 4 位加法器所需要的进位信号 C_{i+1}、C_{i+2}、C_{i+3}、C_{i+4}。三级门电路的延时要比多级行波进位的延时小得多。

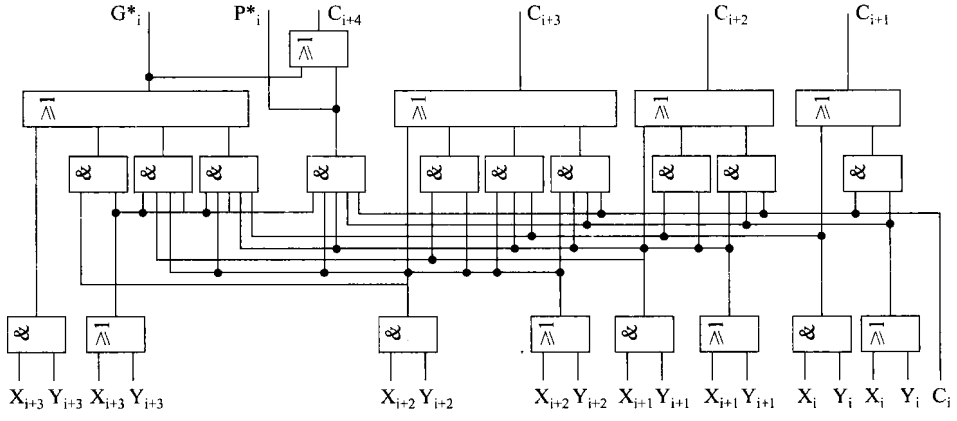


图 3-3　4 位先行进位链电路

以上分析了 4 位先行进位的产生电路。显然，随着位数的增加，这种电路会越来越复杂。因此，在设计加法器时会将多位加法分组，例如以 4 位为一组。将各组串联在一起，即组内采用先行进位方式而组与组之间采用行波进位。这时还会用到前面已经出现的 G^*_i 和 P^*_i 信号。有关此问题的细节将在后面讨论。

5. BCD 数加法器

(1) 8421 BCD 码

在第 2 章中已详细地介绍了 BCD 码。在所描述的几种 BCD 码中，计算机应用最广泛的是 8421 BCD 码。8421 BCD 码只利用了 4 位二进制编码的 0000 至 1001 这 10 种来表示十进制数的 0 至 9。剩余的 6 种：1010、1011、1100、1101、1110、1111 对用于表示十进制数来说是非法的。一旦在定义的 BCD 运算中出现这 6 种编码，结果一定是错误的。

在使用 BCD 码时，若用一个字节(8 位二进制数)表示两位 BCD 数，即高 4 位表示一位 BCD 数，低 4 位表示一位 BCD 数。则此字节所表示的数称为压缩 BCD 数；若一个字节只表示一位 BCD 数，即高 4 位全为 0，只用低 4 位表示一位 BCD 数，则此字节所表示的数称为非压缩 BCD 数。

(2) 加法运算

8421 BCD 码加法运算也可以用多位全加器来实现，在运算过程中有可能产生上面提到的错误结果。为了说明这一问题，请看下面的举例。

【例 3-8】 a. 计算压缩 BCD 数 46＋32＝?

b. 计算压缩 BCD 数 46＋67＝?

解：两题分别相加如下：

a：

$$\begin{array}{rr} & 0100\ 0110 \\ + & 0011\ 0010 \\ \hline & 0111\ 1000 \end{array}$$

b：

$$\begin{array}{rr} & 0100\ 0110 \\ + & 0110\ 0111 \\ \hline & 1010\ 1101 \end{array}$$

由该例可以看到，在 a 的情况下，两 BCD 数相加结果是正确的。在 b 的情况下，两 BCD 数相加结果是错误的。

(3) 校正

为了保证 BCD 数加法运算结果的正确，就必须进行校正。对于一个字节的压缩 BCD 数加法进行校正的法则如下。

① 运算中低 4 位相加的结果＞9 或有由 bit[3]向 bit[4]的进位，则结果加 06H。

② 运算中高 4 位相加(包括由 bit[3]向 bit[4]的进位)的结果＞9 或有由 bit[7]向更高位的进位，则结果加 60H，同时进位为 1 且应看做是相加结果的最高位。

③ 若高低 4 位均不满足上述条件，如例 3-8 中的 a 式，则在校正时加 00H；若同时满足①②两项条件，则在校正时结果加 66H。

以上讨论的是一个字节的压缩 BCD 数加法进行校正的法则。其他运算(减法、乘法、除法)同样需要校正，才能保证结果总是正确的。同样，非压缩 BCD 的运算也需要校正。

上面讨论的是 8421 BCD 码的校正问题，而其他类型的 BCD 码也同样会存在类似的问题。所有这一切此处不再说明。

在设计 CPU 的时候，会在其指令系统中设置一些 BCD 数的各种运算的校正指令，CPU 执行指令时会检测上述法则，根据不同的状态在结果上加上应加的数值。

8421 BCD 加法器及其校正也可以用硬件实现，图 3-4(a)所示的就是带有硬件校正电路的一位 8421 BCD 加法器。图 3-4(b)所示便是一位 BCD 加法器的框图。

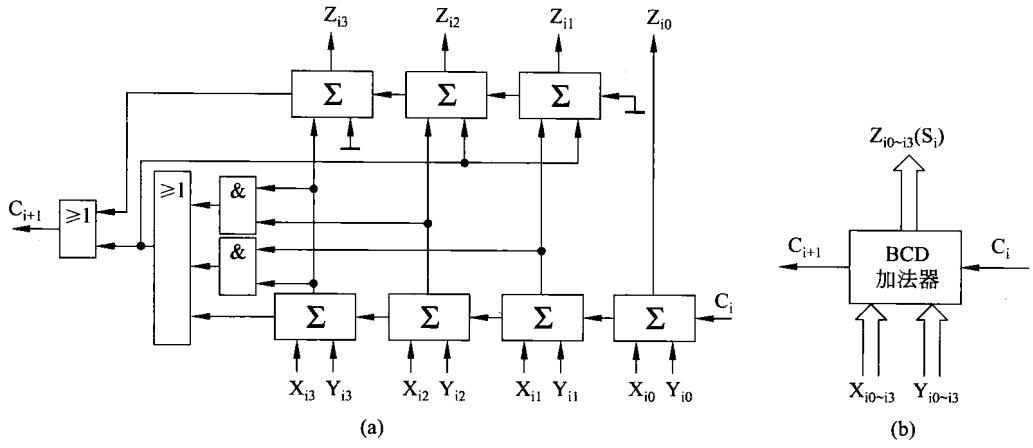


图 3-4　一位 8421 码 BCD 加法器

同样，将多个一位 BCD 加法器串联起来，构成多位行波进位的 BCD 加法器，其框图如图 3-5 所示。

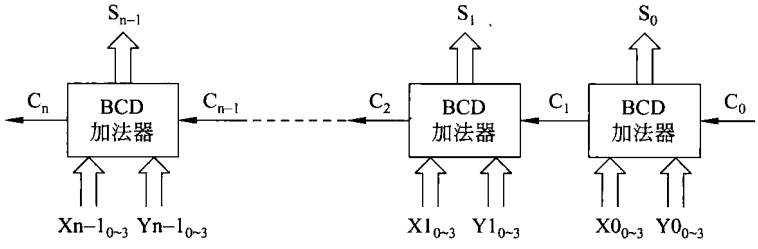


图 3-5　n 位行波进位 BCD 加法器的框图

有关其他类型 BCD 码运算器的硬件实现，这里不再说明。

6. 移码加减运算

(1) 运算法则

关于移码的定义及其特点，在第 2 章中已经做了介绍。由于移码多用在浮点数中表示阶码，因此，在这里仅就定点整数移码的加减运算予以说明。

定点整数移码的加减运算的法则是：①对两移码数求和/差时，首先对这两移码求和/差；②然后，对结果进行修正——将结果的符号位取反。这样，就可以得到正确的结果。下面举例说明：

【例 3-9】 用 8 位二移码表示十进制数的 57 和－35，并且求移码的和与差。

解：十进制数的 57 和－35 用移码表示如下。

$$[57]_{移}=10111001$$

$$[-35]_{移}=01011101$$

求两者之和：

① 首先求$[57]_{移}+[-35]_{移}=10111001+01011101=00010110$；

② 再将符号位取反，从而得到$[57+(-35)]_{移}=10010110$。

求两者之差：

① 首先求$[57]_{移}-[-35]_{移}=[57]_{移}+[[-35]_{移}]_{补}=10111001+10100011=01011100$；

② 再将符号位取反，从而得到$[57-(-35)]_{移}=11011100$。

(2) 移码运算应注意的问题

① 对移码运算的结果需要加以修正，修正量为2^n，即对结果的符号位取反后才是移码形式的正确结果。

② 移码表示中，0有唯一的编码为1000...00。

当出现000...00时(表示：-2^n)，属于浮点数下溢，下溢按机器零处理。

3.1.2 乘法运算

在一些简单的计算机中，乘法运算可以完全用软件来实现。利用计算机所设置加法、移位等指令，编写一段程序完成两数相乘。这样做CPU的硬件结构简单，但实现乘法所用的时间很长，速度很慢。另一种情况是在ALU等硬件的基础上，再适当增加一些硬件构成乘法器。依据这种思路实现乘法，硬件要复杂一些，但速度也比较快。速度最快的是全部由硬件来实现的阵列乘法器。这种方法硬件就更加复杂。可见，可以用硬件来换取速度。前一种方法是软件设计问题，这里不再说明。本小节将讨论后面两种实现乘法的方法。

1. 原码一位乘法运算

(1) 原码一位乘法的法则

假定被乘数X和乘数Y为用原码表示的纯小数(下面的讨论同样适合于纯整数)，分别为：

$$[X]_{原}=X_0.\ X_{-1}X_{-2}\cdots X_{-(n-1)}$$

$$[Y]_{原}=Y_0.\ Y_{-1}Y_{-2}\cdots Y_{-(n-1)}$$

其中X_0、Y_0是它们的符号位。

同时，假定乘积为：$[Z]_{原}=Z_0.\ Z_{-1}Z_{-2}\cdots Z_{-(2n-1)}$

原码一位乘法的法则如下。

① 乘积的符号为被乘数的符号位与乘数的符号位相异或。

② 乘积的绝对值为被乘数的绝对值与乘数的绝对值之积。即

$$[X]_{原}\times[Y]_{原}=(X_0\oplus Y_0)(|X|\times|Y|) \tag{3-14}$$

可见，原码一位乘法可以分别求出积的符号和两乘数绝对值之积，然后拼接在一起，构成乘积。

(2) 原码一位乘法的实现思路

① 手工乘法运算

为了求得绝对值之积，先举例来讨论用手算的过程。

【例3-10】 若$[X]_{原}=0.1101$

$[Y]_{原}=1.1011$，求两者之积。

解：乘积的符号为$0\oplus1=1$

手算过程如下：

$$\begin{array}{r} 1101 \\ \times\ 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline .10001111 \end{array}$$

在上面计算结果小数点左边加上乘积的符号1，即为两者原码积。

在上面的算式中，由于是手算，一次即可把经过左移的4个被乘数加起来，立即获得结果。但在CPU中，由于其内部只有一个ALU，一次只能加一个经左移的(最低位时不左移)被乘数。要加4次才能获得结果。将每次相加的结果称为部分积。

另外，还需要强调的是在上面的手算过程中，根据乘数各位权值不同，每一次使被乘数左移一位加到部分积上。但在实际构成乘法器时，这样做比较麻烦。因此，可以采用被乘数位置不动而使部分积右移。这样做实现起来就更加容易。

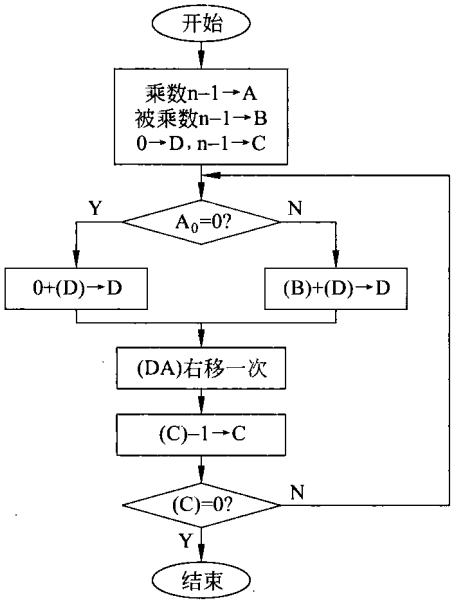


图3-6 绝对值乘法思路框图

② 思路流程

根据上面的分析，可将求绝对值之积的思路描述为如图3-6所示。

设置一个寄存器D，开始置0，运算的中间过程放部分积，最后放结果。用一个寄存器A，开始放乘数，并与D连在一起在运算中间放部分积，最后放结果。从乘数的最低位开始，若为1，则加被乘数到部分积(D连A)。若为0，则部分积加0。部分积右移1位。再检测乘数的次低位，若为1，则加被乘数到部分积(D连A)。若为0，则部分积加0。部分积再右移1位。以此类推，直到乘数各位检测完毕。为了更加明确地描述这种思路，可用图3-6的思路流程图来说明。

(3) 原码一位乘法的具体过程

说明原码一位乘法的运算过程，现仍以例3-10的数据举例。

【例3-11】 $[X]_{原}=0.1101$，$[Y]_{原}=1.1011$，求两者之积。

首先，利用被乘数与乘数的符号，求出乘积的符号为$0 \oplus 1=1$。

第二步求两乘数绝对值之积，求乘积的过程如图3-7所示。

第三步是将乘积的符号与绝对值之积拼接在一起，得到利用原码一位乘法法则所形成的乘积。

(4) 原码一位乘法器框图

根据以上对原码一位乘法的描述，可以相应地画出原码一位乘法器框图，如图3-8所示。

	D				A				A_0	操作
0	0	0	0	0	1	0	1	1		A_0=1,+X
+0	1	1	0	1						
0	1	1	0	1						
0	0	1	1	0	1	1	0	1		→右移一次
+0	1	1	0	1						A_0=1,+X
1	0	0	1	1	1	1	0	1		
0	1	0	0	1	1	1	1	0		→右移一次
0	0	0	0	0						A_0=0,+0
0	1	0	0	1	1	1	1	0		
0	0	1	0	0	1	1	1	1		→右移一次
+0	1	1	0	1						A_0=1,+X
1	0	0	0	1	1	1	1	1		
0	1	0	0	0	1	1	1	1		→右移一次

拼接符号后积为：1. 1 0 0 0 1 1 1 1

图 3-7 例 3-11 的乘法过程

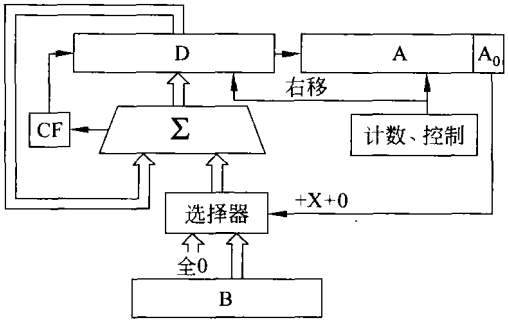


图 3-8 原码一位乘法器框图

由图 3-8 可以看到，当进行乘法运算时，要将寄存器 D 清 0，将被乘数放寄存器 B，乘数放在寄存器 A 中。在运算过程中由寄存器 A 的最低位的状态控制是将 0 还是被乘数加到部分积上。每加一次，在计数和控制电路控制下，将 D、A 两寄存器连在一起右移一次，直到计数为 0，便得到要计算的乘积。

(5) 原码二位乘法

在上述一位乘法中，每次只考虑一位乘数位，对于 n 位乘数(包括一位符号)，则需进行(n−1)次加法和右移。为了提高运算速度，可以一次考虑二位乘数位，每次右移两位。原码二位乘法的法则描述如下。

仍假定被乘数 X 和乘数 Y 为用原码表示的纯小数(下面的讨论同样适合于纯整数)，分别为：

$$[X]_{原}=X_0.\ X_{-1}X_{-2}\cdots X_{-(n-1)}$$

$$[Y]_{原}=Y_0.\ Y_{-1}Y_{-2}\cdots Y_{-(n-1)}$$

其中 X_0、Y_0是它们的符号位。

两位乘数位有 4 种组合：

$Y_{i+1}, Y_i=00$　　对应+0

$Y_{i+1}, Y_i=01$　　对应+|X|

$Y_{i+1}, Y_i=10$　　对应+2|X|

$Y_{i+1}, Y_i=11$　　对应+3|X|

可以看到，每次考虑二位乘数位与一位乘法相比，多出了+2|X|和+3|X|这两种情况。对于前者，只要将|X|左移一位就是 2|X|，在机器内部实现起来很容易，一次即可完成。但是，+3|X|就需两次完成。如果两次操作才能完成+3|X|运算势必要花更多的时间，这也就失去了二位乘法的意义。

解决问题的办法是：①由于二位乘法的运算过程中部分积每次右移两位，右移后+|X|相当于右移两位前+4|X|。②在二位乘法的运算过程中，若遇到 $Y_{i+1}, Y_i=11$ 时，可在部分积中减去|X|，并使一个触发器 C=1。而后，部分积右移两次，若发现触发器 C=1，则需

在部分积上加|X|(相当右移前+4|X|)。而右移前已先减去|X|,与右移后加上|X|综合在一起,实现了4|X|-|X|=3|X|的目的。

经上述分析,现将原码二位乘法的法则列于表3-2中。

表3-2 原码二位乘法的法则表

Y_{i+1}	Y_i	C	操 作
0	0	0	+0,右移2次,C=0
0	0	1	+|X|,右移2次,C=0
0	1	0	+|X|,右移2次,C=0
0	1	1	+2|X|,右移2次,C=0
1	0	0	+2|X|,右移2次,C=0
1	0	1	-|X|,右移2次,C=1
1	1	0	-|X|,右移2次,C=1
1	1	1	+0,右移2次,C=1

为了说明原码二位乘法的运算过程,举例如下。

【例3-12】 设X=+0.100111,Y=-0.100111,利用原码求积。

解:写出$[X]_{原}=0.100111$,$[Y]_{原}=1.100111$。同时,需要注意尽管是绝对值乘,但在实现中是用加减来完成的。为了实现减法,在过程中需用补码做加法。由于一次右移两位,则需用三个符号位。具体过程如图3-9所示。

符号位	D	A	操作
0 0 0 1 1 1	0 0 0 0 0 0 0 1 1 0 0 1	1 0 0 1 1 1	C=0 -X
1 1 1 1 1 1 0 0 1	0 1 1 0 0 1 1 1 0 1 1 0 0 0 1 1 1 0	0 1 1 0 0 1	C=1 →右移二次 C=1,+2X
0 0 1 0 0 0 0 0 1	0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 1 1 0	0 0 0 1 1 0	C=0 →右移二次 C=0,+2X
0 0 1 0 0 0	0 1 1 1 1 1 0 1 0 1 1 1	1 1 0 0 0 1	C=0 →右移二次

图3-9 例3-12的二位乘法的运算过程

乘积之符号=0⊕1=1

$[X\cdot Y]_{原}=1.0\,1\,0\,1\,1\,1\,1\,1\,0\,0\,0\,1$

乘积的符号位单独处理。而在运算过程中,-|X|=111.011001。

2. 补码一位乘法运算

机器中经常采用补码的形式表示数据,这时用原码乘法进行乘法运算就很不方便。因此,不少计算机采取补码乘法器进行乘法运算。通常有两种常用的补码一位乘法的方法:校正法和布斯法。下面仅介绍布斯(Booth)法。

校正法实现补码一位乘法存在的问题在于乘法的最后需要根据乘数的符号位进行校正,显得不够方便。补码一位乘法的另一种运算方法是由布斯(Booth)夫妇首先提出,称为

布斯法。

(1) 算法法则

布斯法的运算法则描述如下。

假定被乘数 X 和乘数 Y 均为用补码表示的纯小数，分别为：

$$[X]_{补}=X_0.\ X_{-1}X_{-2}\cdots X_{-(n-1)}$$

$$[Y]_{补}=Y_0.\ Y_{-1}Y_{-2}\cdots Y_{-(n-1)}$$

其中，X_0、Y_0是它们的符号位。则布斯法补码一位乘法的算法公式为：

$$[X\cdot Y]_{补}=[X]_{补}[(Y_{-1}-Y_0)2^0+(Y_{-2}-Y_{-1})2^{-1}+(Y_{-3}-Y_{-2})2^{-2}+\cdots+(Y_{-(n-1)}-Y_{-(n-2)})2^{-(n-2)}+(0-Y_{-(n-1)})2^{-(n-1)}] \tag{3-15}$$

在这里同样是只给出结论，不做推导。由式(3-15)可以看到，两补码之积可用多项积之和来实现，而每一项中包含用补码表示的乘数相邻两位之差，即需要求出 $Y_{i-1}-Y_i$的值。同时，在最后一项中需要附加一个 0。此种补码一位乘法方法中，被乘数和乘数连同它们的符号位一并参加运算。

乘数的相邻两位之差会有 4 种情况，再根据乘数位所对应的权值，从而导出运算过程中的 4 种操作如表 3-3 所示。

表 3-3 乘数的相邻两位的操作规律

Y_i	Y_{i-1}	$Y_{i-1}-Y_i$	操　作
0	0	0	+0，右移一次
0	1	1	$+[X]_{补}$，右移一次
1	0	−1	$+[-X]_{补}$，右移一次
1	1	0	+0，右移一次

根据以上分析，布斯法的运算法则描述如下。

① 乘数与被乘数均用补码表示，连同符号位一起参加运算。

② 乘数最低位后增加一个附加位(可用 A_{-1}表示)，初始设定为 0。

③ 从附加位开始，依据表 3-3 所示的操作规律，完成式(3-15)的运算。

实现上述算法的布斯法补码一位乘法的思路框图如图 3-10 所示。

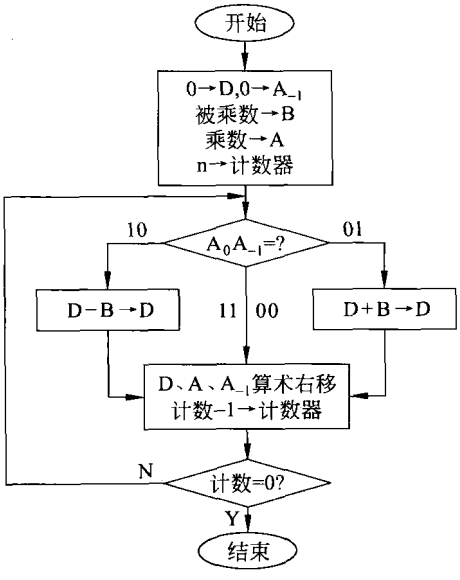


图 3-10 布斯法补码一位乘法流程图

(2) 运算过程

为了说明布斯法补码一位乘法运算的法则，下面通过举例说明运算中运用法则的过程。

【例 3-13】 已知 X＝0.1010，Y＝−0.1101。利用布斯法补码一位乘法求积。

解：首先将两数用补码表示。

$[X]_{补}$=00.1010，$[Y]_{补}$=11.0011，而$[-X]_{补}$=11.0110。下面是布斯法求解的过程，如图 3-11 所示。

符号	D	A	A_{-1}	操作
0 0	0 0 0 0	1 0 0 1 1	0	
1 1	0 1 1 0			$+[-X]_{补}$
1 1	0 1 1 0			
1 1	1 0 1 1	0 1 0 0 1	1	右移一位
0 0	0 0 0 0			+0
1 1	1 0 1 1			
1 1	1 1 0 1	1 0 1 0 0	1	右移一位
0 0	1 0 1 0			$+[X]_{补}$
0 0	0 1 1 1			
0 0	0 0 1 1	1 1 0 1 0	0	右移一位
0 0	0 0 0 0			+0
0 0	0 0 1 1			
0 0	0 0 0 1	1 1 1 0 1	0	右移一位
1 1	0 1 1 0			$+[-X]_{补}$
1 1	0 1 1 1			
1 1	1 0 1 1	1 1 1 1 0		右移一位

图 3-11　例 3-13 的布斯法补码一位乘法过程

$$[X \cdot Y]_{补} = 1.0111110$$

从上述过程可以看到，两补码乘法运算是连同它们的符号位放在一起进行运算的。经过最后一次右移，所要求得的乘积，包括符号位在内就在 DA 连在一起的寄存器中。

(3) 补码一位乘法器(布斯法)框图

根据以上对布斯法补码一位乘法的描述，可以相应的画出乘法器框图，如图 3-12 所示。

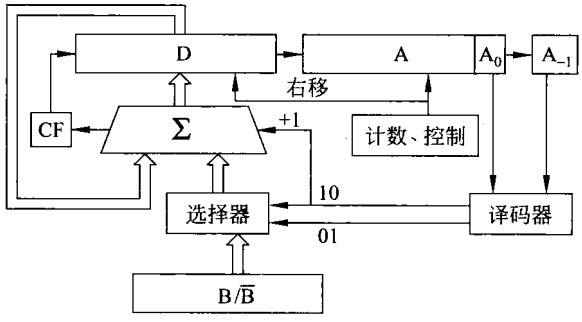


图 3-12　补码一位乘法器(布斯法)框图

由图 3-12 可以看到，运算开始前被乘数放在 B 中(被乘数还可以对各位取反为 $\overline{B}$)，乘数放 A 中，附加位 A_{-1} 和寄存器 D 中放 0。

运算中根据 A_0 和 A_{-1} 的状态决定 +0(不加)、$+[X]_{补}$ 还是 $+[-X]_{补}$。对于 $[-X]_{补}$，就可以如图 3-12 所示，通过被乘数各位取反为 B 再加 1 来获得。

(4) 补码二位乘法

为了提高速度，补码也可以采用二位乘。根据前面补码一位乘法的法则，运算过程中将对 Y_i 与 Y_{i-1} 的状态比较。而二位乘法是将 Y_i 与 Y_{i-1} 的状态比较和 Y_{i-1} 与 Y_{i-2} 的状态比较合在一起进行。由于乘数各位权值不一样，将一位乘法的两步合成一步：

$$2(Y_{i-1}-Y_i)+(Y_{i-2}-Y_{i-1})=Y_{i-1}+Y_{i-2}-2Y_i \tag{3-16}$$

根据式(3-16),可以得出补码二位乘法的运算规律,如表 3-4 所示。

表 3-4 补码二位乘法的运算规律

Y_i	Y_{i-1}	Y_{i-2}	$Y_{i-1}+Y_{i-2}-2Y_i$	操 作
0	0	0	0	+0,右移 2 位
0	0	1	1	$+[X]_{补}$,右移 2 位
0	1	0	1	$+[X]_{补}$,右移 2 位
0	1	1	2	$+2[X]_{补}$,右移 2 位
1	0	0	−2	$+2[-X]_{补}$,右移 2 位
1	0	1	−1	$+[-X]_{补}$,右移 2 位
1	1	0	−1	$+[-X]_{补}$,右移 2 位
1	1	1	0	+0,右移 2 位

由上述分析可以得出补码二位乘法的法则如下。

① 乘数与被乘数均用补码表示,连同符号位一起参加运算。

② 乘数最低位后增加一个附加位(可用 A_{-1} 表示),初始设定为 0。

③ 从附加位开始,依据表 3-4 所示的操作规律,一次检测相邻 3 位决定具体的操作,并每次乘数右移 2 位。

④ 当乘数(包括符号位)为偶数 n 时,右移 2 位的次数为 n/2 次,最后一次只右移 1 位。当乘数(包括符号位)为奇数 n 时,可在乘数最后一位之后添加一个 0,使乘数变为偶数 n+1,右移次数为(n+1)/2,且最后一次只右移 1 位;此时,也可以将乘数增加一个符号位,使乘数变为偶数 n+1,右移次数为[(n+1)/2−1]。

3. 阵列乘法器

(1) 手算及单元电路

在上述乘法运算中,是通过简单的硬件进行多次加法和多次移位来实现乘法。显然,这将难以获得高的运算速度。为了提高运算速度,可以采取类似人工手算的方法。为说明问题,这里列出手算式如下。

设 $X=X_3X_2X_1X_0$,$Y=Y_3Y_2Y_1Y_0$,计算 $X\cdot Y=?$ 列式如下:

$$\begin{array}{cccccccc}
 & & & X_3 & X_2 & X_1 & X_0 \\
 & & \times & Y_3 & Y_2 & Y_1 & Y_0 \\
\hline
 & & & X_3Y_0 & X_2Y_0 & X_1Y_0 & X_0Y_0 \\
 & & X_3Y_1 & X_2Y_1 & X_1Y_1 & X_0Y_1 & \\
 & X_3Y_2 & X_2Y_2 & X_1Y_2 & X_0Y_2 & & \\
X_3Y_3 & X_2Y_3 & X_1Y_3 & X_0Y_3 & & & \\
\hline
Z_6 & Z_5 & Z_4 & Z_3 & Z_2 & Z_1 & Z_0
\end{array}$$

从上式中可以看到,其中 X_i Y_i 是与运算,而结果 Z_i 则是与的结果求和。每一个相与求和操作可以用一个如图 3-13 所示的称为基本乘加单元的电路来实现。

为了求得某一位乘积,基本乘加单元需对本单元上的 X_i Y_i 进行与运算,再加上上一单元给出的部分积,将相加的结果作为部分积加到下一单元,而将进位传至其前面的单元。

(2) 绝对值(无符号数)阵列乘法器

利用上面的手算式的结构及乘加单元电路可以方便地实现绝对值(无符号数)阵列乘法器,这种阵列乘法器的结构框图如图 3-14 所示。

在图 3-14 中,每一个小框即为一个基本的乘加单元,这些基本单元按照图上所示的方式连接,结构上非常类似手算算式的结构,能够完成上面手算算式中的那些乘加功能,获得两数的乘积。

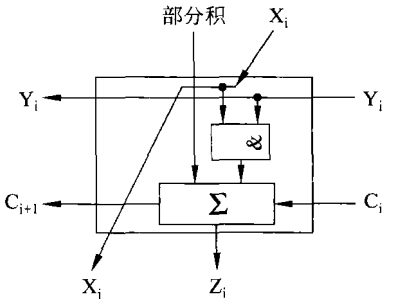


图 3-13 基本乘加单元框图

(3) 带符号数的阵列乘法器

在绝对值阵列乘法器的基础上,很容易实现带符号数的阵列乘法器,其基本思路是先求被乘数与乘数的绝对值,然后进行绝对值乘法,最后再根据被乘数与乘数的符号,决定最后乘积的符号。

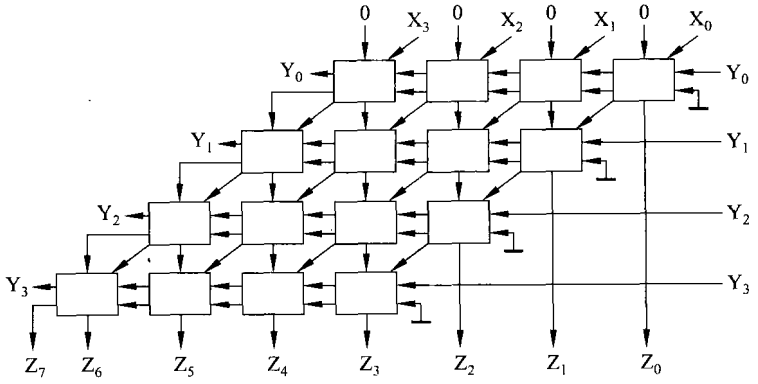


图 3-14 定点无符号数阵列乘法器

为了说明带符号数的阵列乘法器,将一个简单求补电路画在图 3-15 上。

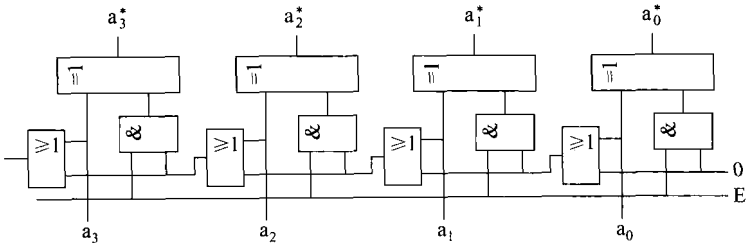


图 3-15 求补电路

由图 3-15 可以看到,当控制端 E 为低电平时,输出与输入相同;当 E=1 时,实现求补。可见,只要用带符号数的符号位加到控制端 E 上,即可求得该符号数的绝对值。

框图 3-16 表示,在进行带符号数乘法之前,先将被乘数和乘数求绝对值(也就是对负数求补),再进行绝对值阵列乘,最后要利用被乘数和乘数的符号位,对绝对值乘积求补。这样,便可获得正确的结果。

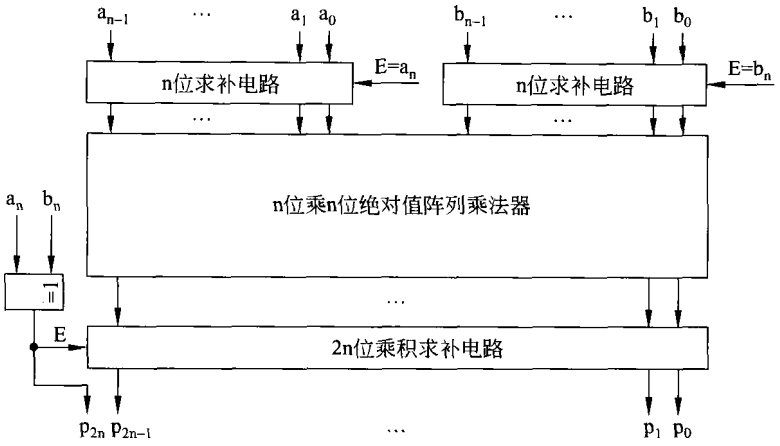


图 3-16 n+1 位带符号数阵列乘法器框图

3.1.3 除法运算

定点除法运算同样可用原码实现也可以用补码实现。在实现除法过程中，应注意除数不能为 0，而且还要保证相除所得的商是可以表示的。

1. 原码除法运算

(1) 原码除法的法则

原码除法的法则应包括如下几点。

① 除数≠0。定点纯小数时，|被除数|<|除数|；定点纯整数时，|被除数|>|除数|。

② 与原码乘法类似的是原码除法商的符号和商的值也是分别处理的，商的符号等于被除数的符号与除数的符号相异或。

③ 商的值就等于被除数的绝对值除以除数的绝对值。

④ 最后将商的符号与商的值拼接在一起就得到原码除法的商。

下面先以手算绝对值除法为例，观察除法的过程，从而导出原码除法的运算方法。

【例 3-14】 设 X=0.1011，Y=0.1101，求 X÷Y=？

解：由题意可知，假定被除数 X 和除数 Y 均为正数。商的符号也为正。

两数绝对值除法的手算过程如下。

在上述手算过程中，需注意如下几点。

① 除法是通过逐次减除数来实现的，也就是被除数(或是余数)每次减去右移一次后的除数，然后决定上商。但在实际构成除法器时，采用的是保持除数的位置不动，而每次余数左移一次。

② 在手工运算过程中，是通过人的眼睛和大脑来判断余数是否够减除数(不需要减过之后进行判断)，以决定商上 0 还是上 1。而在 CPU 中，则必须减过之后方能判断余数是否够减，当发现不够减时，余数已经减掉了除数。因此，在下次操作之前必须恢复余数。这就有下面要介绍的恢复余数法。

```
          0.1101
      ------------
1101 ) 10110
       1101          X/Y=0.1101
       ----
       10010
        1101         余数=0.0111×2^-4
       -----
       010100
         1101        商的符号=0⊕0=0
       ------
         0111
```

(2) 恢复余数法

如上所述，利用恢复余数法实现原码除法遵从前面提到的法则。对于定点纯小数，其过程是符号位单独处理。被除数左移一位，减除数，若够减，上商为1；若不够减，上商为0，同时加除数——恢复余数。余数左移一位，减除数，若够减，上商为1；若不够减，上商为0，同时加除数——恢复余数。重复上面的过程直到除尽或精度达到要求。最后，拼接商符得到商，即可获得除法的结果。下面举例说明恢复余数法做除法的过程。

【例 3-15】 若被除数 X=－0.10001011，除数 Y=0.1110　试利用原码恢复余数法求商及余数。

解：该例满足|X|<|Y|，且|Y|≠0。

写出$[X]_{原}=1.10001011$，$[Y]_{原}=0.1110$。

商符=1⊕0=1。绝对值除法过程如图 3-17 所示。

符号	被除数(余数)								商	操作
0 0	1	0	0	0	1	0	1	1	0	
0 1	0	0	0	1	0	1	1	0		左移一位
1 1	0	0	1	0						−|Y|
0 0	0	0	1	1	0	1	1	0	1	够减，商为1
0 0	0	1	1	0	1	1	0	1		左移一位
1 1	0	0	1	0						−|Y|
1 1	1	0	0	0	1	1	0	1	0	不够减，商为0
0 0	1	1	1	0						+|Y|
0 0	0	1	1	0	1	1	0	1	0	恢复余数
0 0	1	1	0	1	1	0	1	0		左移一位
1 1	0	0	1	0						−|Y|
1 1	1	1	1	1	1	0	1	0	0	不够减，商为0
0 0	1	1	1	0						+|Y|
0 0	1	1	0	1	1	0	1	0	0	恢复余数
0 1	1	0	1	1	0	1	0	0		左移一位
1 1	0	0	1	0						−|Y|
0 0	1	1	0	1	0	1	0	0	1	够减，商为1

图 3-17　例 3-15 的恢复余数法过程

在除的过程中，−|Y|是利用补码加法来实现的。所得的商为商符与绝对值除所得的结果拼接在一起，即商=1.1001；余数的符号与被除数一致，过程中所得余数为绝对值，需将被除数的符号加上去，余数$=1.1101\times2^{-4}$。

恢复余数法做除法最大的缺点是不同的被除数和除数，其除的过程不规范，何时需恢复余数是不相同的，实现起来不便于控制。因此，恢复余数法在计算机中并不常用，而由此法

可演变出另一种更方便的计算方法，这就是加减交替法。

(3) 加减交替法

① 加减交替法的运算法则

为了说明加减交替法的原理，再来回顾恢复余数法做除法。假定第 i 次余数减除数(用 B 表示)得的余数为 R，当 R＜0 时，应恢复余数，即(R＋B)。然后左移一次，即 2(R＋B)。接下来进行下一次(第 i＋1 次)余数减除数，也就是 2(R＋B)－B＝2R＋B。

从上面的分析可以看到，若第 i 次余数减除数的余数 R＜0，这时不再加除数来恢复余数，而只是将其左移一次，变为 2R，到下次，即第 i＋1 次运算时将其加上除数，也就是 2R＋B 即可。这就可以不再用加除数的方法来恢复余数，若发现哪次余数减除数不够减，只需在下次加上除数即可。加减交替法的运算法则描述如下。

a) 若余数 R≥0，则商上 1，左移一次，减除数。

b) 若余数 R＜0，则商上 0，左移一次，加除数。

② 加减交替法的运算过程

下面通过具体的例子说明加减交替法的运算过程。

【例 3-16】 若 X＝－0.10001011，Y＝0.1110　试利用原码加减交替法求商及余数。

解：写出$[X]_{原}$＝1.10001011，$[Y]_{原}$＝0.1110。商符＝1⊕0＝1。

绝对值除的过程如图 3-18 所示。

符号	被除数(余数)								商	操作
0 0	1	0	0	0	1	0	1	1	0	
0 1	0	0	0	1	0	1	1	0		左移一位
1 1	0	0	1	0						－|Y|
0 0	0	0	1	1	0	1	1	0	1	R≥0，商为1
0 0	0	1	1	0	1	1	0	1		左移一位
1 1	0	0	1	0						－|Y|
1 1	1	0	0	0	1	1	0	1	0	R＜0，商为0
1 1	0	0	0	1	1	0	1	0		左移一位
0 0	1	1	1	0						＋|Y|
1 1	1	1	1	1	1	0	1	0	0	R＜0，商为0
1 1	1	1	1	1	0	1	0	0		左移一位
0 0	1	1	1	0						＋|Y|
0 0	1	1	0	1	0	1	0	0	1	R≥0，商为1

图 3-18　例 3-16 的加减交替法过程

通过上面的运算过程，可以得到$[X \div Y]_{原}$＝1.1001，余数＝1.1101×2^{-4}。显然，利用加减交替法所得的结果与恢复余数法是一致的。

③ 加减交替法除法器框图

如前所述，原码加减交替法做除法是符号与数值运算分别进行。下面图 3-19 给出了绝对值(或无符号数)除法的硬件框图。

利用图 3-19 所示的框图，将被除数(绝对值)放在 D、A 寄存器中，将除数放在 B 寄存器中。在进行加减交替做减法时，要用到补码加运算。实际上就是将除数取反加 1 再加到余数上。够减时符号为 0；不够减时符号为 1；符号取反即为商。由此，可大致了解加减交替法除法器的硬件构成。

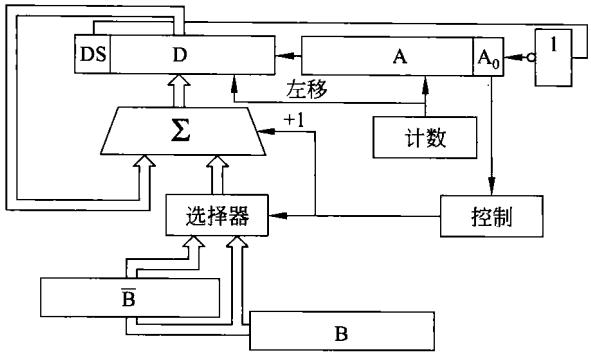


图 3-19 加减交替法除法器框图

2. 补码除法运算

与乘法运算的情况类似，有时也会遇到补码除法。

(1) 补码除法法则

同样，假定进行定点纯小数的补码除法运算，其先决条件是除数≠0，且被除数的绝对值＜除数的绝对值。

补码除法的法则相对要复杂一些，现描述如下。

① 首先，如果被除数与除数同号，被除数减除数；如果被除数与除数异号，被除数加除数。运算的结果均称为余数。

② 若余数与除数同号，上商为 1，余数左移一次，下次用余数减除数操作求商；若余数与除数异号，上商为 0，余数左移一次，下次用余数加除数操作求商。

③ 重复②直至除尽或达到精度要求。

④ 商修正。在除不尽时，通常可用商的最低位恒置 1 进行修正来保证精度。

(2) 补码除法的思路框图

补码除法的思路框图如图 3-20 所示。

由图 3-20 可以看到，该流程图很好地体现了前面提到的运算法则。从比较被除数及除数开始，利用加减交替的方法求得余数，通过余数与除数符号的比较来决定上商为 0 还是为 1。

最后将末位置 1 进行商的修正。

3. 阵列除法器

前面所提到的除法器都是在加法器的基础上，通过多次加减来实现除法，其运算速度必然受到限制。为了提高速度，可以利用硬件来完成除法运算。下面将介绍最简单的无符号数(绝对值)阵列除法器。

(1) 基本概念

在介绍阵列除法器之前，首先说明两个问题。

① 补码运算的进位

在前面已提到，在无符号数做减法时，为了方便是用被减数加上减数的补码来实现的。

这就存在着当被减数小而减数大时，没有进位（或者说没有借位）；而当被减数大而减数小时，反而有进位（或者说有借位）。

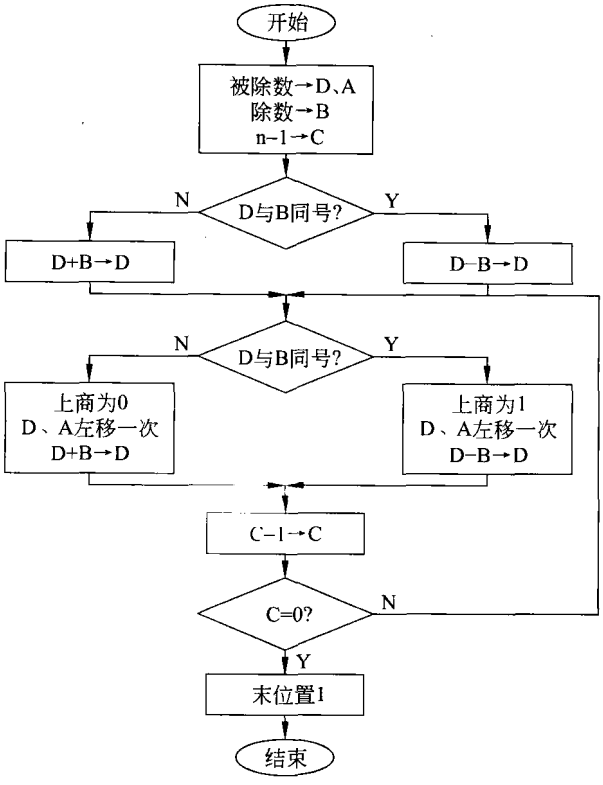


图 3-20　补码除法流程框图

为了说明这种情况，下面举例。

【例 3-17】 无符号数 65－32 和 32－65 两者运算结果的进位情况。

解：无符号数 65 和 32 用 8 位二进制数表示，分别为：

$$65=01000001$$

$$32=00100000$$

在进行减法运算时，可用加补码的方法来实现，它们的补码为：

$$[65]_{补}=10111111$$

$$[32]_{补}=11100000$$

当 65－32 及 32－65 分别列于下：

```
   01000001          00100000
 + 11100000        + 10111111
-----------       -----------
(1)00100001       (0)11011111
   ↗                 ↗
 进位              进位
```

从上面的运算中可以看到，够减时（没有借位），进位为 1；而不够减时进位为 0。

在不同的 CPU 中，这个标志的定义是不一样的，例如 8086、MCS-51 等没有借位进位标

志为0；有借位进位标志为1。而有的则刚好相反，如凌阳16位单片机、ARM处理器等。前者是在做减法时将进位取反后作为进位标志的。在后面的阵列除法器中将用够减时（没有借位），进位为1；而不够减时进位为0这一结论。

② 可控加减单元CAS

可控加减单元如图3-21所示。

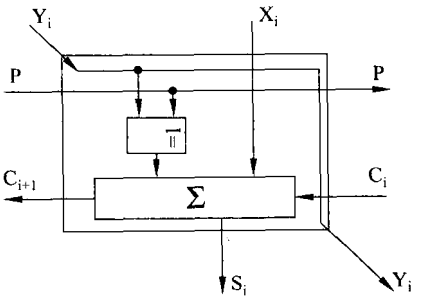


图3-21 可控加减单元CAS

由图3-21可以看到，可控加减单元CAS主要由两部分硬件组成。

一是异或电路，在外部控制信号P的控制下，当P=0时，其输出为Y；当P=1时，其输出为$\overline{Y}$（将输入Y取反）。在下面阵列除法器中，Y作为除数，若将其取反再加1，即可对其求补。

二是一个一位的全加器，实现输入Y、X及前级进位C的全加。

在可控加减单元中，信号Y和控制信号P还直接从本单元中输出加到下一级上。

(2) 无符号数阵列除法器

无符号数阵列除法器如图3-22所示。

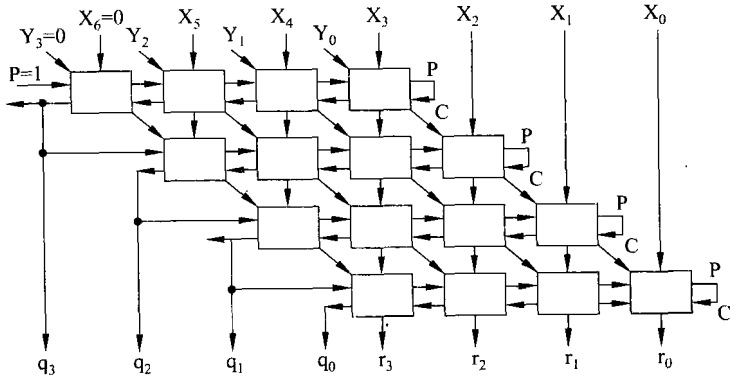


图3-22 由可控加减单元CAS构成的阵列除法器

由图3-22可以看到除法器的基本工作原理。该除法器的被除数加在$X_6 \sim X_0$上，并使$X_6=0$以保证结果正确。除数加在$Y_3 \sim Y_0$上，且使Y_3为0。

开始第一行的P=1，保证将除数取反，P=1又加在第一行最后一个CAS的进位输入端，从而实现了对除数的求补。因此，阵列的第一行就实现了被除数减除数。若够减，则进位为1，商应为1。根据上面的分析，此进位即可为商，即$q_3=1$。相反，若被除数减除数。不够减，则进位为0，商应为0，故$q_3=0$。

下面各行是除数右移一位（相当于余数左移一位），再根据上次运算余数的正负，即够减，则进位为1（上商为1）——本行P=1，做减法，余数减除数；若不够减，则进位为0（上商为0）——本行P=0，做加法，余数加除数。加或减的结果进位即为商。各行完成加减交替法除法运算。

利用图3-22阵列除法器，得到商$=q_3\ q_2\ q_1\ q_0$；余数$=r_3\ r_2 r_1\ r_0$。最后提醒读者，若纯小

数运算，应满足 X＜Y。

3.2 算术逻辑单元

在本节中将简单介绍已开发出的算术逻辑单元电路，对于有关电路的设计将留最后一章再做介绍。

3.2.1 单元电路

前面已经提到了全加器，用它再加上其他电路可以构成加法器、乘法器、除法器等算术运算部件。而与门、或门、异或门、反相器等组合在一起可以构成逻辑运算部件。作为一种单元电路这里不再说明。在此仅介绍其他有关电路。

1. 寄存器

在计算机系统中或在构成 CPU 时，都会用到寄存器。前人已提供许多种类的寄存器。工程上经常用到的有集成 D 锁存器及带有三态输出的 D 锁存器。

(1) 8D 锁存器

这种锁存器有多种型号，这里仅给出 74LS273 作为例子。

74LS273，它是由 8 个 D 触发器集成在一块芯片中构成。其引线及功能表分别如图 3-23 及表 3-5 所示。

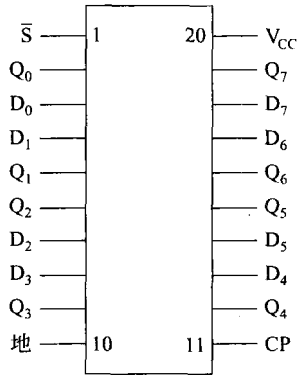


图 3-23 74LS273 锁存器引线

表 3-5 74LS273 功能表

S	CP	D_X	Q_X
0	X	X	0
1	↑	1	1
1	↑	0	0

74LS273 是将 8 个 D 触发器集成在一块芯片中，输入为 D_0～D_7 而输出为 Q_0～Q_7。同时将 8 个 D 触发器的脉冲触发端在内部连在一起，外部上升沿触发端为 CP。

由于该锁存器的输出是二态的，没有第三态，即(高阻)状态。因此，单独的锁存器只能作为输出接口。它不能单独作为输入接口，因为当它作为输入接口时，必然引起数据总线竞争。在使用时应特别注意。

(2) 带有三态门输出的锁存器

带有三态门输出的锁存器也有多种。在这里给读者介绍一种带有三态门输出的锁存器芯片 74LS374，这也是经常使用的芯片，其引线图如图 3-24 所示。

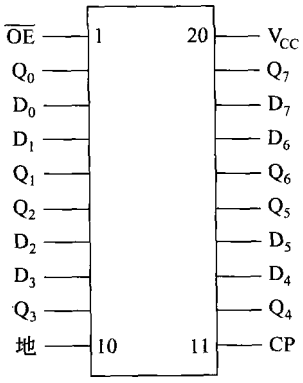


图 3-24　74LS374 的引线

表 3-6　74LS374 真值表

D_X	CP	$\overline{OE}$	Q_X
1	↑	0	1
0	↑	0	0
X	X	1	Z

由真值表 3-6 可以看到，74LS374 与 273 的最重要的差别在于 374 的输出 Q 是经 8 个三态门输出到芯片外部。这些三态门是由输出允许信号 $\overline{OE}$控制的，当 $\overline{OE}=0$ 时，三态门导通；而当 $\overline{OE}=1$ 时，三态门高阻。

由于 374 中既集成了锁存器又集成了三态门。因此，它既可以作为输出接口又可以作为输入接口使用。

2. 移位寄存器

有多种型号的移位寄存器可供人们选用。图 3-25 所给出的是移位寄存器 74198 的引线。其功能表如表 3-7 所示。

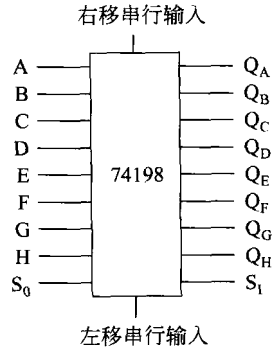


图 3-25　移位寄存器 74198 引线

由表 3-7 可以看到，该移位寄存器可以并行输入并行输出，也可以串行输入并行输出；既可以左移又可以右移；只要稍做工作它就可以实现移位及循环移位；它还可以作为寄存器使用。详细情况不再说明。

表 3-7　移位寄存器 74198 功能表

输入							输出				
清除	模式		时钟	串行		并行	Q_A	A_B	…	Q_G	Q_H
	S_1	S_0		左	右	A…H					
L	X	X	X	X	X	X	L	L	…	L	L
H	X	X	L	X	X	X	Q_{A0}	Q_{B0}	…	Q_{G0}	Q_{H0}
H	H	H	↑	X	X	A…H	A	B	…	G	H
H	L	H	↑	X	H	X	H	Q_{An}	…	Q_{Fn}	Q_{Gn}
H	L	H	↑	X	L	X	L	Q_{An}	…	Q_{Fn}	Q_{Gn}
H	H	L	↑	H	X	X	Q_{Bn}	Q_{Cn}	…	Q_{Hn}	H
H	H	L	↑	L	X	X	Q_{Bn}	Q_{Cn}	…	Q_{Hn}	L
H	L	L	X	X	X	X	Q_{A0}	Q_{B0}	…	Q_{G0}	Q_{H0}

3.2.2 算术逻辑单元 ALU

前人将算术运算、逻辑运算以至于移位功能结合到一起，研制出算术逻辑单元(ALU)集成电路。其中，中规模集成电路 74181 是一种 4 位的算术逻辑单元电路。

1. 74181 的引线

74181 的引线如图 3-26 所示。74181 有正逻辑工作的芯片，也有负逻辑工作的芯片，图 3-26 所画的是工作在正逻辑的。74181 的运算功能列于表 3-8 中。

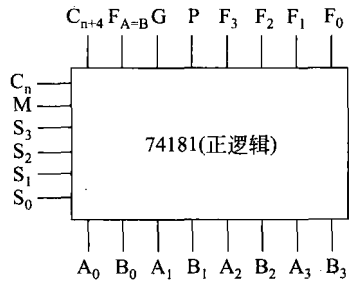


图 3-26 4 位 ALU74181 引线

由表 3-8 可以看到，74181 有很多功能，在芯片上的控制信号的控制下，可实现各种算术运算和逻辑运算，这里不再做仔细说明。在此要强调的是 74181 内部已集成了在本章前面图 3-3 所示的先行进位电路，使 74181 的 4 位加法的延迟时间很小。同时，为了提高级联工作的速度，74181 可以输出 G^*_{i+3} 和 P^*_{i+3}，即芯片上的 G 和 P。

表 3-8 74181 的运算功能

操作选择	运算功能		
$S_3\ S_2\ S_1\ S_0$	M=1 逻辑运算	M=0 算术运算	
		Cn=1 无进位	Cn=0 有进位
0 0 0 0	$F=\overline{A}$	F=A	F=A 加 1
0 0 0 1	$F=\overline{A+B}$	F=A+B	F=(A+B)加 1
0 0 1 0	$F=\overline{A}B$	$F=A+\overline{B}$	$F=(A+\overline{B})$加 1
0 0 1 1	F=0	F=减 1	F=0
0 1 0 0	$F=\overline{AB}$	F=A 加 $A\overline{B}$	F=A 加 $A\overline{B}$ 加 1
0 1 0 1	$F=\overline{B}$	F=(A+B)加 $A\overline{B}$	F (A+B)加$\overline{A\overline{B}}$加 1
0 1 1 0	$F=A\oplus B$	F=A 减 B 减 1	F=A 减 B
0 1 1 1	$F=A\overline{B}$	F=$A\overline{B}$ 减 1	$F=A\overline{B}$
1 0 0 0	$F=\overline{A}+B$	F=A+AB	F=A 加 AB 加 1
1 0 0 1	$F=\overline{A\oplus B}$	F=A+B	F=A 加 B 加 1
1 0 1 0	F=B	$F=(A+\overline{B})$加 AB	$F=(A+\overline{B})$加 AB 加 1
1 0 1 1	F=AB	F=AB 减 1	F=AB
1 1 0 0	F=1	F=2A	F=A 加 A 加 1
1 1 0 1	F=A+B	F=(A+B)加 A	F=(A+B)加 A 加 1
1 1 1 0	$F=A+\overline{B}$	$F=(A+\overline{B})$加 A	F=(A+B)加 A 加 1
1 1 1 1	F=A	F=A 减 1	F=A

2. 级联工作

单独一片 74181 只能实现 4 位的算术或逻辑运算。显然，4 位运算的精度太低，但

74181 可以级联工作，用两片 74181 可以构 8 位的 ALU，3 片 74181 可以构 12 位的 ALU。图 3-27 表示了用 4 片 74181 构成的片间串行进位的 16 位 ALU。

在图 3-27 中，每片 74181 内部采用的是先行进位方式，速度很快。但是芯片与芯片之间却是串行(行波)进位方式，速度比较慢。也有人称这种级联方式为组内先行进位，组间串行进位。

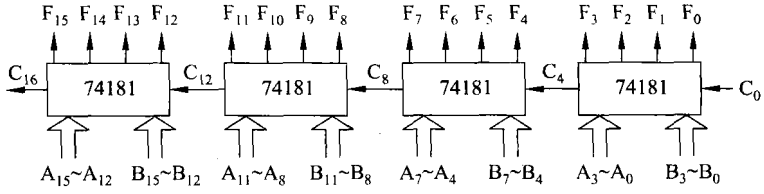


图 3-27 由 74181 构成组间串行进位的 16 位 ALU

为了提高级联 ALU 的运算速度，前人已研制出并(先)行进位链集成电路 74182。对于该芯片不再做更多地说明，只是强调 74182 可以利用 C_0 及 74181 提供的 G 和 P 便能先行产生图 3-27 连接所需要的进位信号，从而提高整个 ALU 的速度。利用 74182 构成的片(组)内、片(组)间均并行进位的 ALU 框图如图 3-28 所示。

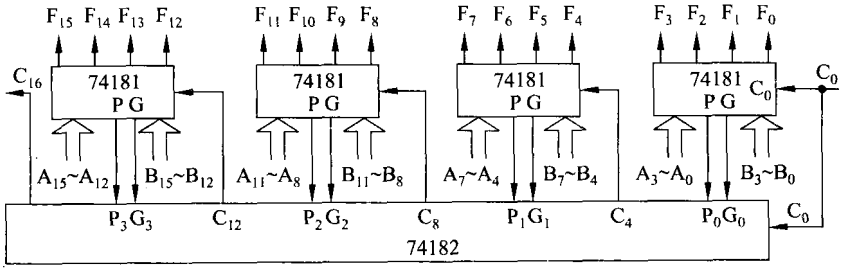


图 3-28 由 74181 和 74182 构成组内组间均并行进位的 16 位 ALU

3.2.3 运算器的结构

1. 三种基本结构

根据运算器内部总线与构成运算器的基本部件的连接情况，运算器有三种基本结构形式：单总线结构、双总线结构及三总线结构。它们分别由图 3-29 的(a)、(b)、(c)、(d)表示。

除了如图 3-29 所表示的结构外，还有其他类似的连接形式，不再说明。在这里要强调的是：

(1) 总线总是分时工作的，即在任何时刻只允许传送一个部件的信号。也就是说任何时候只允许一个器件将其信号输出加到总线上。多于一个器件必然引起总线竞争。

(2) 同一个功能部件一次只能做一件事。如 ALU 可以完成加、减、与、或等多种功能。但某一时刻只能完成一种功能，做加法时不可能同时做与运算。

(3) 在双总线及三总线结构的运算器中需要多端口器件。图 3-29 所给出的运算器就是依据上述所强调的原则工作的。

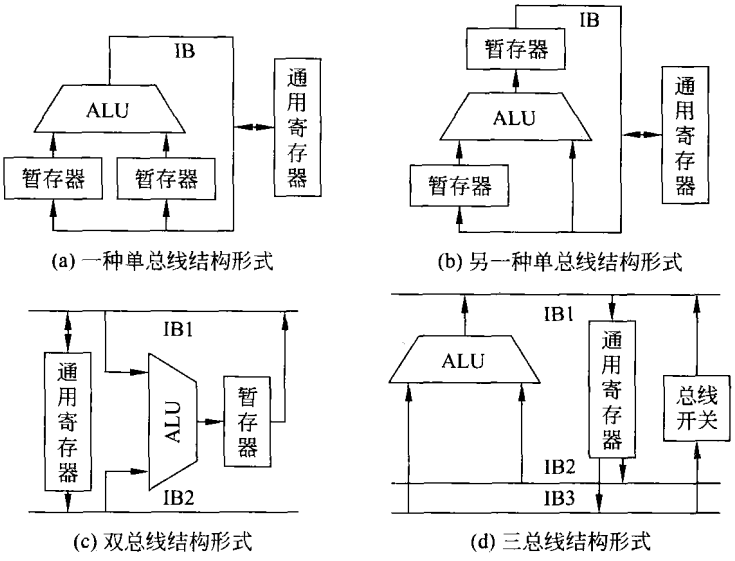


图 3-29 运算器的三种基本结构

2. 计算机简单框图

一种简化的计算机框图如图 3-30 所示。

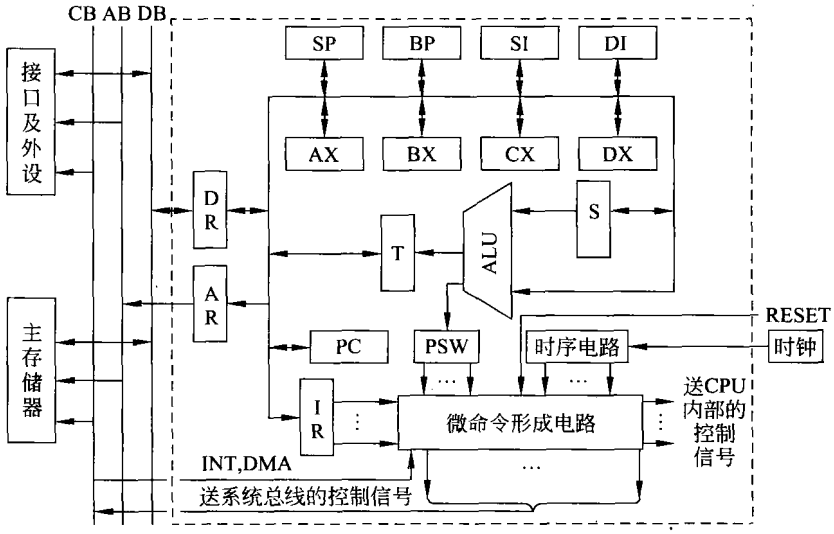


图 3-30 某计算机简化框图

在图 3-30 中采用的是比较简单的单总线结构，其右上方的一些部件构成运算器。其中 ALU 完成算术逻辑运算，暂存器 T 还可以实现移位及循环操作，S 为暂存器，还有 8 个通用寄存器 AX、BX、CX、DX、SP、BP、SI、DI。

图 3-30 的右下方是后面第 5 章将要讲述的控制器。图 3-30 的左方是计算机的系统总线，主要包括双向数据总线 DB、地址总线 AB 和控制总线 CB。数据和地址分别通过双向数

据寄存器 DR 和地址寄存器 AR 与系统总线相连接。构成计算机的主存储器、接口及外设就接在系统总线上。

图 3-30 中，用虚线框起来的部分就是 CPU。它是由运算器、寄存器、内部总线和控制器构成。

3.3 浮点运算

有关浮点数的定义，规格化等问题已在第 2 章中做了详细说明，在本节中只描述浮点数的各种运算。

3.3.1 加减运算

1. 浮点数加减法的运算法则

设两个浮点数：$X=Mx \cdot 2^{Ex}$

$Y=My \cdot 2^{Ey}$

实现 X±Y 运算的法则即过程如下 4 步。

(1) 对阶

一般情况下，两浮点数的阶码不会相同，也就是说两数的小数点没有对齐。与所熟悉的十进制小数加减一样，在进行加减运算前需将小数点对齐。这就是对阶，只有当两者的阶码相同时才能进行尾数的加减运算。

对阶的原则是小阶对大阶，也就是将小阶码变成大阶码。具体做法是小阶码每增加 1，其相应的尾数右移一位，直到增大到与大阶码相同。这样的原则在对阶时丢失的是尾数的低位，造成的误差很小。若是大阶对小阶将丢失尾数的高位，会导致错误的结果。

(2) 尾数进行加(减)运算

在对阶之后，尾数进行加(减)运算。实际上只做加法就可以了，因为减法可以用加法来实现。在做减法时，只要先将减数求补，然后与被减数相加即可。

(3) 规格化

在进行加减运算后，其结果有可能是一个非规格化数。如果结果是非规格化数，则需要规格化。这时可能会出现如下两种情况。

① 左规

如果运算结果尾数为 11.1XX...X 或者是 00.0XX...X 时，规格化需将尾数左移。每左移一位，阶码减 1，直到使尾数成为规格化数为止。这一过程称为左规。

在左规时，尾数每左移一位，阶码减 1，必须同时判断阶码是否减到比所能表示的阶码还小。一旦出现这种情况，阶码将无法表示这样小的阶码，这就是下溢出。发生下溢出时可认为结果为 0。

若阶码(包括 1 位符号位)用 m 位补码整数表示，它能表示的最小阶码为 -2^{m-1}。如果左规使阶码小于 -2^{m-1}，则发生下溢出。

② 右规

如果在尾数加(减)运算时,结果(尾数)发生溢出,即结果出现 10.XX...X 或者出现 01.XX...X 时,表明尾数出现溢出。但是,并不表明整个浮点数结果溢出。

出现上述情况,可将尾数右移一次,阶码加 1,此过程称为右规。在浮点数加减运算时,右规最多 1 次。

右规使阶码加 1,有可能使阶码超出所能表示的最大范围,如上定义为($2^{m-1}-1$)。如果出现这种情况,结果将无法表示,这就是上溢出。一旦发生上溢出,可认为结果为∞。

可见,浮点数的溢出与否是由阶码决定的。

(4) 舍入处理

在对阶及规格化时需要将尾数右移,右移将丢掉尾数的最低位,这就出现舍入的问题。在进行舍入时,通常可采用下面的方法。

① 截(尾)断法

此法最简单,就是将需丢弃的尾数低位丢弃。

② 末位恒置 1 法

无论尾数右移丢弃的是 0 还是 1,此法将保证要保留的尾数的最低位永远为 1。

③ 0 舍 1 入法

当尾数右移丢弃的是 1 时,要保留的最末位加 1;当尾数右移丢弃的是 0 时,要保留的最末位不变。

这种方法误差较小。但当遇到 01.111...11 这种需在规的尾数时,采用此法会再次使尾数溢出。遇到这种情况可采用截尾法。

2. 浮点数加(减)法流程

假定两浮点数 X、Y 相加(或相减)的结果为浮点数 Z,浮点数加(减)法流程如图 3-31 所示。

3. 浮点数运算实例

下面举例说明浮点数加法的过程。

【例 3-18】 两浮点数为:

$$X=0.110101\times 2^{-010}$$

$$Y=-0.101010\times 2^{-001}$$

求两数之和及差。

解:设两浮点数阶码为 4 位,用补码表示。尾数用 8 位,均用双符号位补码表示。则两数可表示为:

$$[X]_{浮}=1110 \quad 00.110101$$

$$[Y]_{浮}=1111 \quad 11.010110$$

(1) 对阶

求阶差:$[\Delta E]_{补}=[E_X]_{补}+[-E_Y]_{补}=1110+0001=1111$,即 X 的阶码比 Y 的阶码小。

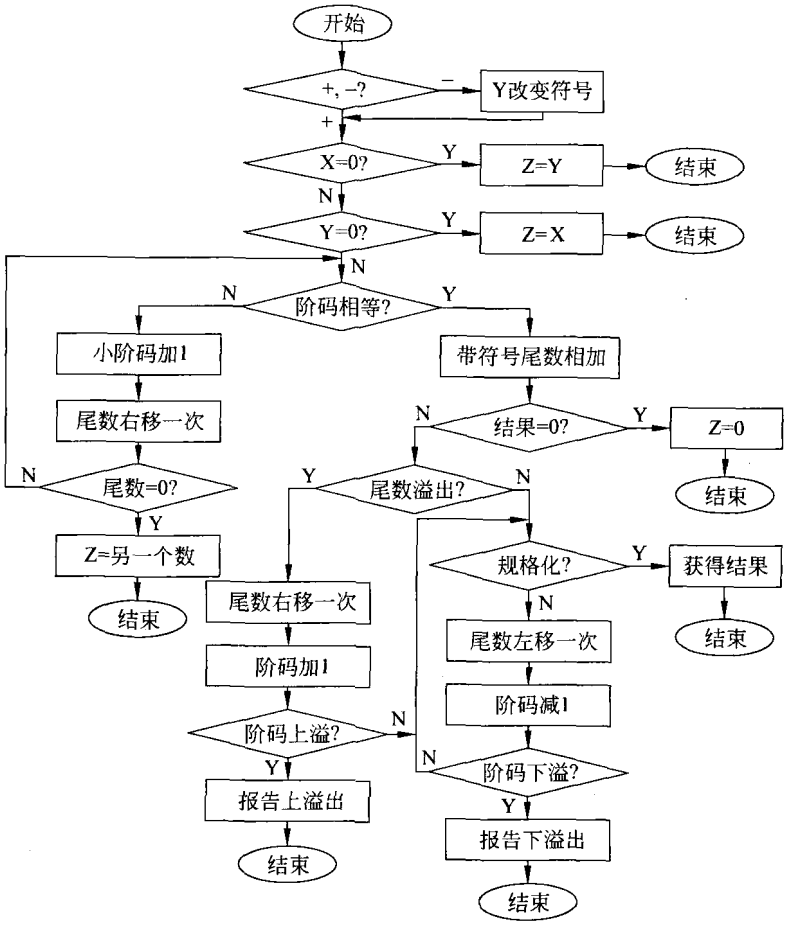


图 3-31　浮点数加(减)法运算流程

因此,X 尾数右移一次,使两者阶码相同。这时的 X 为$[X]'_{浮}=1111;00.011011$,采用的是 0 舍 1 入法。

(2) 尾数求和:

$$\begin{array}{r} 00.011011 \\ +\ 11.010110 \\ \hline 11.110001 \end{array}$$

尾数求差:

$$\begin{array}{r} 00.011011 \\ +\ 00.101010 \\ \hline 01.000101 \end{array}$$

(3) 规格化

从相加的结果可以看到结果为一个非规格化尾数。需将尾数左移 2 位,变为 11.000100。同时,阶码减 2,则阶码变为 1101。

最后,得到两数相加结果为$[X+Y]_{浮}=1101;11.000100$。

同样,从两者相减的结果看到尾数也不是规格化数。此时需右规,将尾数右移一次,变为 00.100011,采用的是 0 舍 1 入法。同时,阶码需加 1,则阶码为 0000。得到两数相减结果为$[X-Y]_{浮}=0000;00.100011$。

3.3.2 浮点乘除运算

1. 浮点乘法运算

(1) 浮点乘法运算的过程

设两个浮点数：$X=Mx\cdot 2^{Ex}$

$Y=My\cdot 2^{Ey}$

该两浮点数相乘为 $Z=(Mx\cdot My)2^{Ex+Ey}$。可以看到，两浮点数相乘之积的阶码为两乘数阶码之和，乘积的尾数为两乘数尾数之积。

浮点乘法的运算过程如下。

① 参加乘法运算的两浮点数一定是规格化数，且不为0。只要有一个乘数为0，则乘积必为0。

② 求乘积的阶码，即 $Ez=Ex+Ey$。并判断积的阶码是否溢出。

当积的阶码大于所定义浮点数最大阶码，即 $Ez>Emax$ 时，上溢出。一旦发生上溢出，则乘积将无法表示。若积的阶码小于所定义浮点数最小阶码，即 $Ez<-Emax$ 时，则下溢出。发生下溢出时，乘积可用0表示。当发生溢出时，尤其是上溢，应重新定义浮点数或对两乘数做出限制。

③ 两乘数的尾数相乘。

两尾数相乘可用本章3.1节所描述的方法进行。

④ 规格化乘积的尾数。

假定尾数为n位补码(其中含1位符号)，其规格化正数范围为 $+1/2\sim+(1-2^{-(n-1)})$；而规格化负数的范围为 $-(1/2+2^{-(n-1)})\sim-1$。

前面已经提则，两乘数的尾数均为规格化数，根据上述规格化数的范围，两者之积的绝对值肯定不小于1/4。因此，乘积的尾数如果需要左规，也只需要1次左移，便可使积之尾数变为规格化数。

同样，两乘数的尾数均为规格化数，两者之积有可能为+1，即两个-1相乘。而+1不是规格化数，因此，乘积的尾数如果需要右规，也只需要1次右移，便可使积之尾数变为规格化数。

规格化中，若进行右规，则应注意采用哪种舍入的算法。最终可以获得两浮点数的乘积。

(2) 浮点乘法的流程

根据上述浮点数乘法的过程，可以画出浮点乘法流程框图如图3-32所示。

2. 浮点除法运算

(1) 浮点除法运算的过程

设两个浮点数：$X=Mx\cdot 2^{Ex}$

$Y=My\cdot 2^{Ey}$

该两浮点数相除为 $Z=(Mx\div My)2^{Ex-Ey}$。可以看到，两浮点数相除的结果的阶码为被除数的阶码减去除数的阶码；商的尾数为被除数的尾数除以除数的尾数。

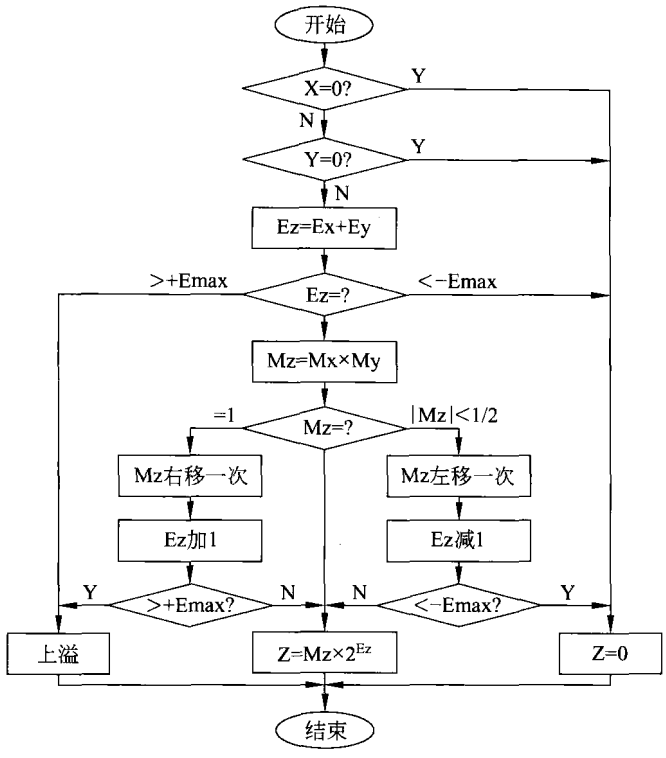


图 3-32　浮点乘法流程框图

浮点除法的运算过程如下。

① 参加除法运算的两浮点数一定是规格化数。除数不能为 0，否则肯定出错。若被除数为 0，则商必为 0。

② 求商的阶码，即 Ez=Ex−Ey。并判断积的阶码是否溢出。

当商的阶码大于所定义浮点数最大阶码，即 Ez>Emax 时，上溢出。一旦发生上溢出，则商将无法表示；若商的阶码小于所定义浮点数最小阶码，即 Ez<−Emax 时，则下溢出。发生下溢出时，商可用 0 表示。当发生溢出时，尤其是上溢，应重新定义浮点数或对被除数除数做出限制。

③ 被除数的尾数除以除数的尾数。

在这里，由于进行的是浮点数运算，只要求两尾数是规格化数。并不要求被除数的绝对值小于除数的绝对值。除法可用本章前一节所描述的方法进行。

④ 规格化商的尾数。

根据上面提到的规格化数的范围，两者相除时，可能出现 1/2 除以 −1 的情况，此时的商为 −1/2，这不是规格化数，此时的尾数需要左规，但只需要 1 次左移，便可使商的尾数变为规格化数。

同样，当被除数的尾数绝对值大于除数的尾数的绝对值时，会产生整数商。此时商的尾数需要右规，也只需要 1 次右移，便可使商的尾数变为规格化数。

规格化中，若进行右规，则应注意采用哪种舍入的算法。最终可以获得两浮点数的商。

(2) 浮点除法的流程

根据上述浮点数除法的过程，可以画出浮点数除法流程框图如图 3-33 所示。

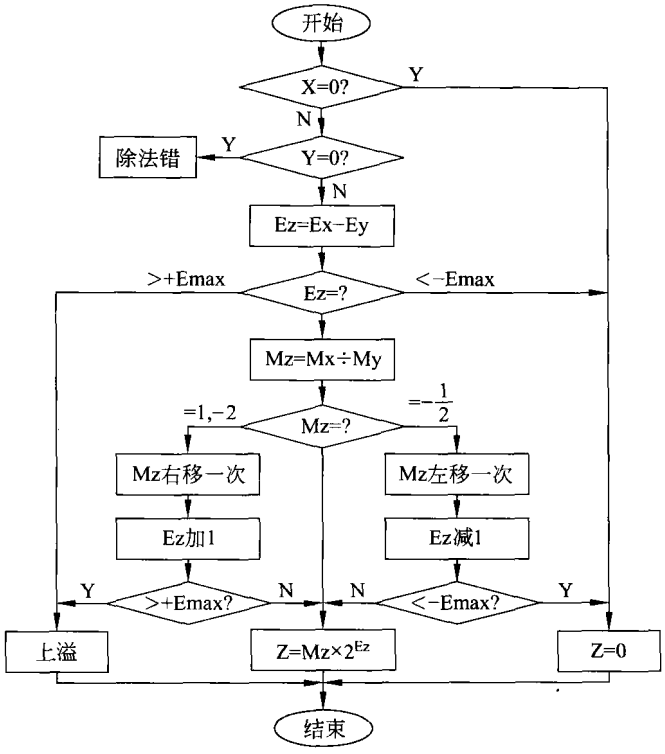


图 3-33 浮点数除法流程框图

3.3.3 浮点数运算的实现

在计算机中实现浮点数运算可以用不同的方法。

1. 软件方法

在一些比较简单的微型机、单片机中，已设置有加减法指令，其内部有算术逻辑单元。可以通过编写程序的方法来实现浮点运算。在 20 世纪 80 年代笔者曾做过这样的工作。利用浮点数运算，在 Z80 上实现 FFT 的谱分析功能。这种方法速度慢、工作量很大。今天应尽可能避免使用这种方法。

2. 配专用浮点数处理器

在许多没有浮点数处理能力的处理器之外，另外配上浮点数处理器。例如，在 8086 构成的微型机中可以配置浮点协处理器 8087。80286 上可配上 80287、80386 上配置 80387 等。这样就可以提高计算机的浮点数处理能力。当然，今天的高档处理器早已把浮点协处理器集成在处理器芯片内部，如从 80486DX 之后全是这样的。

将来在设计一套计算机系统时，若采用的处理器不支持浮点运算，而系统又需要进行浮

点运算时，可以考虑在此系统中配置独立的浮点协处理器。

3. 在处理器中设置浮点数运算部件

如果有浮点数运算的需求，在设计处理器(CPU)时，可以将浮点数运算器放进处理器中。这样的处理器在设计制造出来之后就能实现浮点数运算的功能。

显然，后两种方法要付出硬件上的代价，但浮点数运算的速度必然比软件实现要来得快。

4. 浮点运算的流水线处理

有关流水线的问题将在本书后面的章节中详细讲述。在这里不做介绍。

习　题

1. 简要说明运算器的功能。
2. 说明采用单符号位检测溢出的方法。
3. 说明采用双符号位检测溢出的方法。
4. 根据前面图 3-4 说明 BCD 加法器的工作原理。
5. 说明浮点运算中溢出的处理办法。
6. 某微型机字长 16 位，若采用定点补码整数表示数值，最高 1 位为符号位，其他 15 位为数值部分，则所能表示的最小正数和最大负数各为多少？
7. 在进行定点原码乘法运算时，乘积的符号位是由被乘数的符号位和乘数的符号位进行何种运算来获得的？
8. 用字长为二进制 16 位，定点数补码纯小数所能表示的最大的正数及最小的负数各为多少？
9. 当两个同符号的数相加或异符号的数相减，所得结果的符号位 SF 和进位标志 CF 进行何种运算为 1 时，表示运算的结果产生溢出？
10. 若浮点数的阶码用移码表示，尾数用补码表示。两规格化浮点数相乘，最后对结果规格化时，右规的右移位数最多为几位？左规的左移位数最多为几位？为什么？
11. 分析下面图 3-34 所示的框图并回答下列问题。

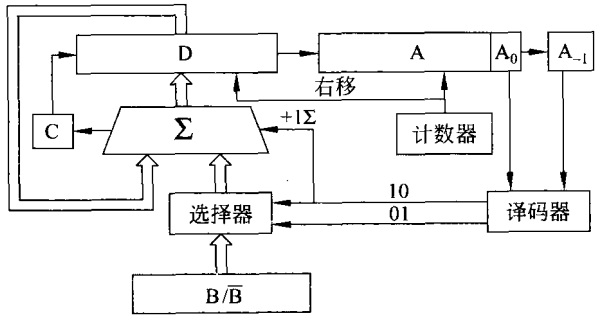


图 3-34　习题 11 附图

该框图所完成的功能是什么？在运算开始时，D、A、B 寄存器中分别存放的是什么？

12. 若机器字长为 8 位，定点小数表示。已知 X =－0.10110，Y =0.10010。

(1) 求 $[X]_{补}$、$[Y]_{补}$ 和 $[-Y]_{补}$。

(2) 用变形补码计算 $[X+Y]_{补}$ 和 $[X-Y]_{补}$，并判断结果是否溢出。

13. 下面图 3-35 是一位算术逻辑单元(ALU)的结构框图。根据此图判断控制信号 S_2、S_1、S_0 为何种状态时，$F_i=0$。S_2、S_1、S_0 为何种状态时，$F_i=X_i+Y_i$。

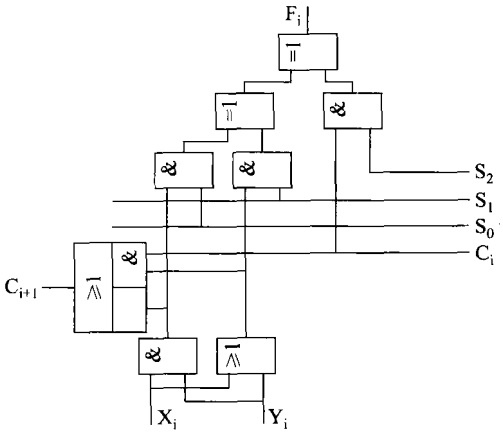


图 3-35 习题 13 附图

14. 已知 $[X]_{补}=11001100$，$[Y]_{补}=10101001$ 分别为纯小数，且最高位为符号位，求两者之和为多少？并说明(有/无)溢出。

15. 若已知 $2[X]_{补}=1.0101001$，$1/2[Y]_{原}=1.01011000$，试利用变形码计算 $[X]_{补}+[Y]_{补}=$？并判断结果有无溢出？

16. 若已知 $1/2[X]_{补}=1.0101001$，$2[Y]_{原}=1.0101100$，试利用变形码计算 $[X]_{补}+[Y]_{补}=$？并判断结果有无溢出？

17. 试依据下列条件，用补码计算 $[X]_{补}+[Y]_{补}=$？并判断结果有无溢出？

(1) X=0.01001　　Y=－0.10111

(2) X=0.10010　　Y=0.11000

(3) X=－0.01101　　Y=0.00101

(4) X=－0.11011　　Y=－0.10010

18. 将下列十进制数变换为 6 位移码表示，其中最高 1 位为符号位，其余 5 位为数值位，计算 $[X+Y]_{移}=$？并判断结果有无溢出？

(1) X=18　　Y=15

(2) X=－22　　Y=－15

19. 将下面十进制数用 8421 码表示，进行运算并校正。

(1) 88+99

(2) 27+15

20. 根据题中的数据分别用原码 1 位乘及布斯(Booth)法求乘积，并写出计算步骤。

(1) X=－-0.1101　　Y=＋0.0110

(2) X=−0.1110　　Y=−0.1101

21. 试利用 74181 芯片和 74182 芯片构成 32 位的 ALU，画出连接框图。

22. 分别用原码加减交替法和补码加减交替法完成 X 除以 Y 的运算并写出运算步骤。

(1) X=−0.10101　　Y=+0.11011

(2) X=+0.1001110001　　Y=−0.10101

23. 定点运算器的内部总线结构有三种形式，下面的描述_____是对应三总线结构的。

A. 执行一次操作需要三步

B. 在此运算器中至少需要设置两个暂存器

C. 在运算器中的两个输入和一个输出上至少需要设置一个暂存器

D. 在运算器中的两个输入和一个输出上不需要设置暂存器

24. 浮点数中，用补码表示的 n 位(包括 1 位符号位)规格化负尾数的取值范围可表示为多少？

25. 浮点数字长 16 位，阶码 7 位(其中包括阶符 1 位)，数符 1 位，尾数 8 位。若阶码用移码表示，尾数用补码表示。该浮点数所能表示的数值范围为多少？

26. 设浮点数字长 12 位，阶码 5 位(含 1 位符号)，尾数 7 位(含 1 位符号)。阶码用移码、尾数用补码表示。将下列各数按浮点数相加减的步骤计算 X±Y。

(1) $X=11/16\times2^{-4}$　　$Y=35/64\times2^{-3}$

(2) $X=+0.101101\times2^{-11}$ B　　$Y=(-0.100101)\times2^{-1}$B

27. 试分析在浮点数进行算术运算(加、减、乘、除)时，在什么情况下会产生上溢出。

28. 图 3-36 为某一运算器的简化框图，其中 A、B 既有寄存器又有多路选择器的功能，M 有图中所标功能及多路选择功能，ALU 为算术逻辑单元，R0 和 R1 为通用寄存器。图中粗线为数据通路，是一种单总线结构。试写出下列运算的步骤。

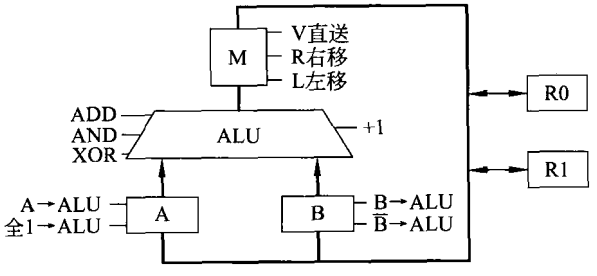


图 3-36　习题 28 附图

(1) (R0)+2(R1)→R0

(2) 2(R0)−(R1)→R0

(3) (R0)−1→R0

(4) (R0)XOR(R1)→R0

29. 一个可完成加、减、乘、除 4 种定点运算的运算器结构如图 3-37 所示。图中 A，B，C 为三个寄存器，并且 A，B 具有联合左，右移位的功能，$\sum$为可完成加，减运算的加法器。箭头及连接线标明了数据流通的方向。

分析该图，说明在加法运算开始前，两个加数及加法完成后的和应分别放入的寄存器是哪几个？

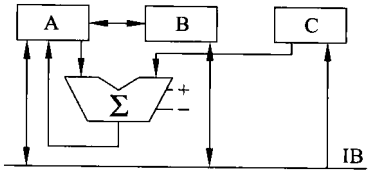


图 3-37　习题 29 附图